LeafOS Resident Project Stack

Quick Start `v0.5.0`

LeafOS

Updated 2026-07-21

Current default: attach the local stack to a project with `leafctl live PATH`. The stack means all downloaded and locally available models in this LeafOS instance. Models propose; CPU-side policy executes and validates.

1. Check The Host

From PowerShell in the taskpack directory:

```
.\bin\leafctl.ps1 doctor
.\bin\leafctl.ps1 versions
.\bin\leafctl.ps1 provider-stack check
```

The provider check validates the configured llama.cpp executable, local GGUF model, Vulkan route, endpoint, and lifecycle settings. It does not download a model.

2. Open A Project

```
.\bin\leafctl.ps1 live C:\R\MyProject
```

LeafOS performs bounded read-only intake, derives an objective when none is supplied, creates or reattaches a persistent version-2 run, starts the resident supervisor, and opens the terminal interface.

Inspect without starting work:

```
.\bin\leafctl.ps1 live C:\R\MyProject --inspect --json
```

Create a minimal Catan2 project:

```
.\bin\leafctl.ps1 live C:\Games\catan2 \
  --new --template catan2 --yes --budget 64
```

Intake does not add LeafOS control files to an existing target. Run state stays in the LeafOS runs directory.

3. Give Instructions

Inside the active window, plain language is enough:

```
:improve
:improve add deterministic replay
:again
:make state changes easier to inspect
```

Every instruction converges on one lifecycle:

```
INTAKE -> PLAN -> APPROVE -> EXECUTE -> VALIDATE -> CHECKPOINT/REPORT
```

The provider creates a bounded structured proposal. The CPU inlet validates paths, commands, approval, dependencies, attempts, execution, and acceptance. A failure preserves evidence and may create one bounded repair task.

4. Control The Resident Stack

```
:mode quiet
:mode auto
:targets cpu 80 gpu 90
:budget 64m
:pause
:resume
:drain
```

The utilization values are useful-work targets, not load quotas. Recent input, responsiveness, RAM, VRAM, temperature, and provider health can reduce or close dispatch. Pressing `q` closes only the interface. Drain finishes accepted work. Stop uses a confirmed typed request.

5. Reconnect And Observe

```
.\bin\leafctl.ps1 tui --run active
.\bin\leafctl.ps1 loop status active
.\bin\leafctl.ps1 loop attach active --after 0
.\bin\leafctl.ps1 resident status active --json
```

Run directories retain the work order, queue, state, events, controls, resident decision, provider artifacts, command output, validation, checkpoint, telemetry, and report required to reconnect without conversation memory.

6. Verify

```
.\bin\leafctl.ps1 test visual --plain
.\bin\leafctl.ps1 test visual \
  --match provider --live-stack --plain
.\bin\leafctl.ps1 catan2-resident \
  --contract-only --iterations 3
```

The current discovery suite contains 203 tests. The live-stack demonstration is explicitly real-provider evidence. Contract and fixture tests remain clearly labeled and do not make that claim.

Authority And Privacy

The TUI is a structured projection and typed control surface, not a shell or a second scheduler. Models cannot approve themselves, write files directly, or turn output into evidence. LeafOS displays plans, progress, decisions, tools, validation, and accepted results. It does not expose or persist private model chain-of-thought.

Next Documents

Document	Purpose
`DOCUMENTATION_MAP.md`	Current authority and historical boundaries.
`ARCHITECTURE.md`	Process, data, provider, and UI boundaries.
`AGENTIC_CLI.md`	Instruction normalization and lifecycle.
`RESIDENT_STACK_USAGE.md`	Resource policy, budgets, recovery, and soaks.
`MEDIUM_MOE_MODEL_POLICY.md`	Model candidates, SSD truth, and qualification gates.
`TUI_USAGE.md`	Pages, keys, snapshots, and control transport.

Current Acceptance State

The resident implementation, contract profile, 203-test observatory, native TUI, bounded live-provider demonstration, and real four-minute Catan2 profile pass. The real eight-minute and 64-minute soaks remain open evidence. A profile is not complete merely because its command exists.

Optional Surfaces

- `oneshot` creates a handoff bundle; it does not run a project.
- `dashboard` displays the optional node/cluster subsystem.
- `node`, `nodes`, and `task` distribute explicit SSH jobs.
- Early agent-plan and graph commands remain compatibility paths.

Historical work orders and reports preserve their original terminology as dated evidence. Use the Documentation Map when a historical command conflicts with this guide.